Category A Problems

5 problems · ~5 per session, per the facilitation guide's pacing · full 5-step walkthrough with alternative approaches and real complexity discussion · OOP, Classes and Objects — Week 3 · Homework Assignment

F1. From Procedural Mess to a Working Library Fine System

Scenario

The library assistant currently tracks five overdue books using five sets of parallel variables copied from an old script. The librarian has just asked for two more things by end of day: flag which books are overdue by more than 14 days, and compute the total fine collected across all of them.

Task

- Design a class BookIssue with fields title, borrowerName, and daysOverdue, all set through a constructor.
- Add an instance method fineAmount() that returns daysOverdue * 5 (Rs 5 per day) when daysOverdue > 0, else 0.
- Add an instance method isSeverelyOverdue() returning true when daysOverdue is greater than 14.
- Add a static method totalFineCollected(BookIssue[] issues) that sums fineAmount() across an array of BookIssue objects — this belongs to the class as a whole, not to any one book.
- In main, build an array of five BookIssue objects, print each one's title and overdue status, then print the total fine using the class name.
- In a code comment, justify why totalFineCollected is static while fineAmount is not.

Suggested method signature(s)

```
double fineAmount()
boolean isSeverelyOverdue()
static double totalFineCollected(BookIssue[] issues)
```

Sample Input / Output

Input	Output
5 books, daysOverdue: *Clean Code 18, Effective Java 5,* *Refactoring 0, DSA Handbook 21,* *Design Patterns 9*	*Clean Code - 18 days - Severely overdue* *Effective Java - 5 days - OK* *Refactoring - 0 days - OK* *DSA Handbook - 21 days - Severely overdue* *Design Patterns - 9 days - OK* *Total fine collected: Rs 265.0*

Concepts covered: *OOP vs procedural design, constructors, instance methods, static utility methods operating over many objects, arrays of objects.*

F2. Extending Employee Without Touching It

Scenario

HR already relies on a tested Employee class. The company now wants a ManagerEmployee and an InternEmployee in addition to plain Employee, without Employee.java ever being reopened and edited again.

Task

- Define Employee with private empId, empName, and salary; a constructor; and a method getSalary().
- Define ManagerEmployee extends Employee, adding a private double teamBonus set through its own constructor, plus a new method effectiveSalary() that returns getSalary() + teamBonus. Do not modify Employee to make this work.
- Define InternEmployee extends Employee, adding a private double stipendCap set through its own constructor, plus a new method effectiveSalary() that returns whichever is smaller: getSalary() or stipendCap.
- In main, create one of each of the three employee types, and print each one's pay — using instanceof to decide which extra behaviour to invoke for each type.

Suggested method signature(s)

```
class ManagerEmployee extends Employee { double effectiveSalary() }
class InternEmployee extends Employee { double effectiveSalary() }
```

Sample Input / Output

Input	Output
Plain: salary 40000 *Manager: salary 70000, teamBonus 8000* *Intern: salary 12000, stipendCap 10000*	*Plain employee pay: Rs 40000.0* *Manager effective pay: Rs 78000.0* *Intern effective pay: Rs 10000.0*

Concepts covered: *Inheritance (extends), encapsulation, extensibility without editing tested code, instanceof-based dispatch.*

F3. Object References, Null Safety, and a Mutating Method

Scenario

The campus parking allocation service occasionally has no slot left to hand back, and a recent bug crashed the whole allocation run because nobody checked for that case. Build — and prove — a version that never crashes.

Task

- Define ParkingSlot with fields slotNo, capacity, occupiedCount, and a method allot(String vehicleNo) that fills a spot if one is free.
- Write a static method ParkingSlot findAvailableSlot(ParkingSlot[] slots) that returns the first slot with occupiedCount < capacity, or null if every slot is full.
- Write a static method safeAllot(ParkingSlot[] slots, String vehicleNo) that calls findAvailableSlot, checks the result for null before touching it, and either allots the vehicle or prints a clear “no slots available” message — with zero risk of a NullPointerException, regardless of input.
- In main, call safeAllot() twice: once against an array containing an available slot, and once against an array where every slot is already full, to prove both paths behave correctly.
- In a comment, explain why passing the ParkingSlot array into these methods does not copy the slots themselves.

Suggested method signature(s)

```
static ParkingSlot findAvailableSlot(ParkingSlot[] slots)
static void safeAllot(ParkingSlot[] slots, String vehicleNo)
```

Sample Input / Output

Input	Output
Slots: A1 (3/4), A2 (5/5) *safeAllot(slots, "TN09AB1234")*	*TN09AB1234 allotted to slot A1*
Slots: A1 (4/4), A2 (5/5) *safeAllot(slots, "TN09AB1234")*	*No slots available for TN09AB1234*

Concepts covered: *Object references vs copies, null handling, avoiding NullPointerException, static utility methods over object arrays.*

F4. Designing the Instance/Static Boundary for a Library Membership System

Scenario

A junior developer's first draft of LibraryMember marks every single field static "to make it easier to access from anywhere." The sample run below shows the disaster that follows. Reproduce the bug, then redesign the class properly and prove the fix with the same test.

Task

- First reproduce the broken version: class LibraryMember { static String name; static String memberId; static int booksIssued; ... } and create two members one after another.
- In a comment, explain — for each field — exactly why marking it static is wrong here.
- Redesign the class with the correct split: name, memberId, and booksIssued as instance fields; libraryName and memberCount as static fields.
- Add a constructor that derives memberId automatically from memberCount, an instance method printMemberCard(), and a static method printTotalMembers().
- In main, run the broken version first and show how creating the second member silently overwrote the first member's data. Then run the corrected version and show both members now keep fully independent data.

Suggested method signature(s)

```
// broken: static String name; static String memberId; static int booksIssued;
// fixed: instance name/memberId/booksIssued, static libraryName/memberCount
void printMemberCard()  |  static void printTotalMembers()
```

Sample Input / Output

Input	Output
Broken version: *new LibraryMember("Aditi", ...)* *new LibraryMember("Rohan", ...)* *then print both members' names*	*Rohan* *Rohan* *(Aditi's data was overwritten — both members now show "Rohan")*
Fixed version: same two members created	*Aditi | LM-1001* *Rohan | LM-1002* *Total members: 2*

Concepts covered: *Instance vs static design decisions, constructors, why static fields are shared, debugging a real static-misuse bug.*

F5. Capstone: A Small HR + Parking Allocation Mini-System

Scenario

Combine everything from this week into one small but complete mini-system that a company's HR office could plausibly plug in tomorrow.

Task

- Reuse Employee and ManagerEmployee extends Employee (as in F2), and ParkingSlot with the null-safe allotment process (as in F3).
- Write a class CompanyEmployeeRecord that ties an employee's name, empId, an Employee (or ManagerEmployee) object, and a parking slot assignment together as fields of one object — demonstrating that an object's fields can themselves be objects.
- Add a static int totalRecords counter, incremented in the CompanyEmployeeRecord constructor, and an instance method fullProfile() that prints the employee's name, effective pay, and slot number in one line — printing “no parking assigned” instead of a slot number when the slot reference is null.
- In main, build a small system of three employee records, allot parking to only two of them (leaving the third unallotted on purpose), and print each record's fullProfile(), followed by CompanyEmployeeRecord.totalRecords.

Suggested method signature(s)

```
class CompanyEmployeeRecord { String name; String empId; Employee employee; ParkingSlot slot; }
String fullProfile()
```

Sample Input / Output

Input	Output
3 records; parking allotted to 2 of them	*Divya | Pay: Rs 78000.0 | Slot: A1* *Karan | Pay: Rs 40000.0 | Slot: A2* *Meera | Pay: Rs 10000.0 | Slot: no parking assigned* *Total records: 3*

Concepts covered: *Composition (objects containing objects), encapsulation, inheritance, null-safe field handling, static counters — the whole week, working together.*